

9장 명령어 선택

최광훈
전남대학교

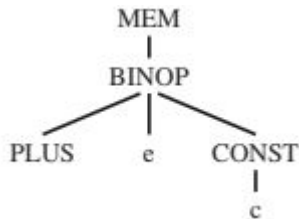
개요

중간 표현(**Tree**) 언어는 각 트리 노드에 하나의 기본 연산만 표현

- 메모리 읽기 또는 쓰기, 덧셈 또는 뺄셈, 조건 분기, 기타 기본 연산

실제 기계 명령어와의 차이

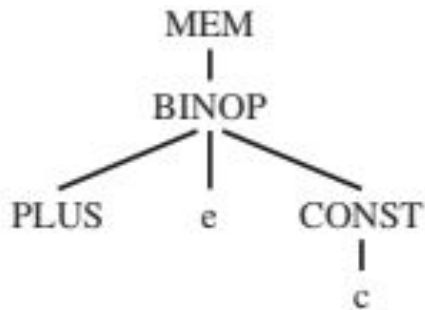
- 종종 이러한 기본 연산 여러 개를 하나의 명령어 안에서 함께 수행하는 형태
- 주소 계산 $e + c$ 하고 해당 주소에서 메모리 읽기



개요

명령어 선택(Instruction Selection)

- 주어진 중간 표현 트리를 실제 기계 명령어로 구현하기 위해
- 적절한 기계 명령어를 선택하는 과정



MIPS 명령어 : `lw r2, c(r1)` -- e의 값 => r1

x86 명령어 : `mov ebx, [eax + c]` -- e의 값 => eax

(레지스터 할당 전 단계에서는
r1, r2, eax, ebx를 임시 변수 temp로 표시함)

개요

트리 패턴(Tree Patterns)

- 기계 명령어는 IR 트리의 일부 조각, 이러한 조각을 트리 패턴이라고 함.
- 명령어 선택 문제: 트리 타일링(tree tiling),
=> IR 트리를 가능한 적은 수의 트리 패턴으로 덮기

개요

아키텍처 예시

(1) TEMP 노드 (레지스터)

(2) 결과를 레지스터에 저장

(3) 결과를 메모리에 저장

각 명령어: 하나 이상의 트리 패턴

Name	Effect	Trees
(1) —	r_i	TEMP
ADD	$r_i \leftarrow r_j + r_k$	
MUL	$r_i \leftarrow r_j \times r_k$	
SUB	$r_i \leftarrow r_j - r_k$	
DIV	$r_i \leftarrow r_j / r_k$	
(2) ADDI	$r_i \leftarrow r_j + c$	
SUBI	$r_i \leftarrow r_j - c$	
LOAD	$r_i \leftarrow M[r_j + c]$	
STORE	$M[r_j + c] \leftarrow r_i$	
MOVEM	$M[r_j] \leftarrow M[r_i]$	

FIGURE 9.1.

Arithmetic and memory instructions. The notation $M[x]$ denotes the memory word at address x .

- +, *와 같은 교환 법칙이 성립하는 연산

- LOAD, STORE처럼 레지스터나 상수가 0일 수 있는 경우

개요

IR 트리 타일링

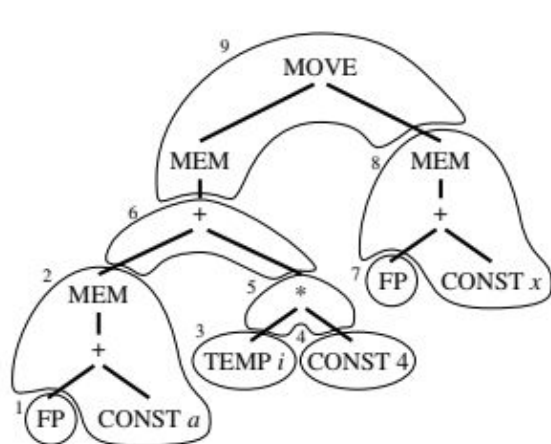
- 트리 기반 중간 표현을 이용한 명령어 선택의 핵심은 **IR** 트리를 타일링하는 것
- 타일은 합법적인 기계 명령어에 대응되는 트리 패턴들의 집합
- 목표는, **IR** 트리를 서로 겹치지 않는 타일들로 완전히 덮는 것
- 각 타일은 하나의 기계 명령어에 대응되고, 타일링 결과가 곧 명령어 열이 됨

개요

IR 트리 타일링 예: $a[i] := x$

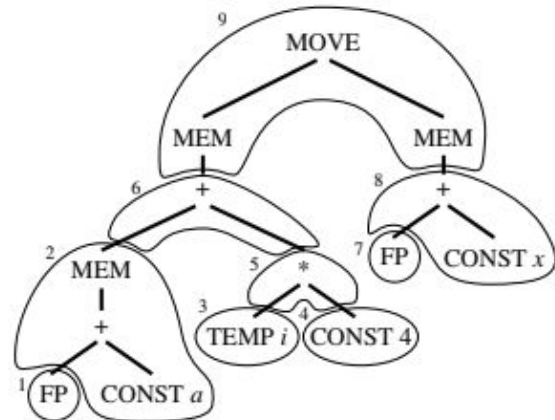
- i 는 레지스터 변수

- a 와 x 는 프레임 변수



2	LOAD	$r_1 \leftarrow M[\mathbf{fp} + a]$
4	ADDI	$r_2 \leftarrow r_0 + 4$
5	MUL	$r_2 \leftarrow r_i \times r_2$
6	ADD	$r_1 \leftarrow r_1 + r_2$
8	LOAD	$r_2 \leftarrow M[\mathbf{fp} + x]$
9	STORE	$M[r_1 + 0] \leftarrow r_2$

(a)



2	LOAD	$r_1 \leftarrow M[\mathbf{fp} + a]$
4	ADDI	$r_2 \leftarrow r_0 + 4$
5	MUL	$r_2 \leftarrow r_i \times r_2$
6	ADD	$r_1 \leftarrow r_1 + r_2$
8	ADDI	$r_2 \leftarrow \mathbf{fp} + x$
9	MOVEM	$M[r_1] \leftarrow M[r_2]$

(b)

FIGURE 9.2. A tree tiled in two ways.

개요

IR 트리 타일링 예: $a[i] := x$

- i 는 레지스터 변수
- a 와 x 는 프레임 변수

가장 단순한 타일링

- 각 타일이 트리 노드 하나만 포함하도록 하는 것

ADDI	$r_1 \leftarrow r_0 + a$
ADD	$r_1 \leftarrow \mathbf{fp} + r_1$
LOAD	$r_1 \leftarrow M[r_1 + 0]$
ADDI	$r_2 \leftarrow r_0 + 4$
MUL	$r_2 \leftarrow r_i \times r_2$
ADD	$r_1 \leftarrow r_1 + r_2$
ADDI	$r_2 \leftarrow r_0 + x$
ADD	$r_2 \leftarrow \mathbf{fp} + r_2$
LOAD	$r_2 \leftarrow M[r_2 + 0]$
STORE	$M[r_1 + 0] \leftarrow r_2$

개요

가장 좋은 타일링은 가장 낮은 비용의 명령어열에 대응

- 명령어 갯수가 가장 적은 명령어열
- 전체 실행 시간이 가장 낮은 명령어열(명령어마다 실행 시간이 다르다면)

각 명령어에 비용을 정의한다면,

- 전역 최적 타일링(**optimum tiling**): 타일들의 비용 합이 가능한 가장 작은 타일링
- 최적 인접 타일링(**optimal tiling**): 지역적으로 더 개선할 수 없는 타일링

: 서로 인접한 두 타일을 하나로 합쳤을 때 더 낮은 비용의 타일이 되는 경우가 없음

- 전역 최적 타일링 => 최적 인접 타일링 (반대 방향 <= 성립 X)

9.1 명령어 선택 알고리즘

CISC vs RISC 타일링

구분	CISC	RISC
이름	Complex Instruction Set Computer	Reduced Instruction Set Computer
명령어 성격	여러 연산을 한 번에 수행	적은 수의 연산만 수행
타일 크기	큼	작음
명령어 비용	다양함	비교적 균일
optimum과 optimal 차이	종종 차이가 있음	보통 거의 없음
필요한 알고리즘	정교한 알고리즘 필요	단순한 알고리즘으로 충분

9.1 명령어 선택 알고리즘

Maximal Munch 방식 명령어 선택

- 가능한 큰 타일을 먼저 선택하는 타일링 방식

알고리즘:

- 트리의 루트에서 시작하여 그 위치에 맞는 가장 큰 타일을 찾고,
- 그 타일로 루트 노드와 루트 근처의 여러 노드를 덮고,
- 아직 덮이지 않은 각 서브트리에 대해 같은 과정을 반복

9.1 명령어 선택 알고리즘

명령어 생성 순서

- **Tree IR**의 아래에서 위 방향으로 생성
- 실제 코드를 출력할 때에는 서브트리에서 생성된 명령어들이 먼저 나오고
- 루트 명령어는 그 뒤에 나옴

9.1 명령어 선택 알고리즘

명령어 선택 과정 예시 (test/Test0.java)

(1) MiniJava 문장 `System.out.println(123);`

(2) Tree IR의 문장으로
변환

```
EXP(  
  CALL(  
    NAME _print,  
    CONST 123)),
```

(3) MIPS 명령어 선택(Assem)

```
li t36, 123  
move $a0, t36  
jal _print
```

(4) 레지스터 할당
(11장/12장)

```
li $v1, 123  
move $a0, $v1  
jal _print
```

- Temp munchExp(CONST 123) => li t36, 123 (식의 값: t36 변수를 리턴!)
- void munchStm (CALL (NAME _print, [...])) => [...]의 값을 담은 변수 t36을 함수 첫 번째 인자 레지스터 \$a0에 저장 하고, _print 함수를 호출

9.1 명령어 선택 알고리즘

명령어 선택 과정 예시

- test/Test0.java ~ test/Test4.java

- 1번~4번까지 과정을 설명

1. MiniJava 문장 -> 2. Tree IR 문장 -> 3. MIPS 명령어 선택(Assem.Instr)

-> 4. 레지스터 할당 (10장, 11장)

9.1 명령어 선택 알고리즘

Java 구현 방식

- munchStm(...) 문장 IR 노드를 처리
- munchExp(...) 식 IR 노드를 처리

9.1 명령어 선택 알고리즘

PROGRAM 9.3. Maximal Munch in Java.

```
munchStm(Stm s)
└─ munchMove(dst, src)
    └─ munchMove(MEM dst, Exp src)
    └─ munchMove(TEMP dst, Exp src)
    └─ munchMove(Exp dst, Exp src)
```

```
void munchMove(MEM dst, Exp src) {
    // MOVE(MEM(BINOP(PLUS, e1, CONST(i))), e2)
    if (dst.exp instanceof BINOP && ((BINOP)dst.exp).oper==BINOP.PLUS
        && ((BINOP)dst.exp).right instanceof CONST)
        {munchExp(((BINOP)dst.exp).left); munchExp(src); emit("STORE");}
    // MOVE(MEM(BINOP(PLUS, CONST(i), e1)), e2)
    else if (dst.exp instanceof BINOP && ((BINOP)dst.exp).oper==BINOP.PLUS
        && ((BINOP)dst.exp).left instanceof CONST)
        {munchExp(((BINOP)dst.exp).right); munchExp(src); emit("STORE");}
    // MOVE(MEM(e1), MEM(e2))
    else if (src instanceof MEM)
        {munchExp(dst.exp); munchExp(((MEM)src).exp); emit("MOVEM");}
    // MOVE(MEM(e1), e2)
    else
        {munchExp(dst.exp); munchExp(src); emit("STORE");}
}

void munchMove(TEMP dst, Exp src) {
    // MOVE(TEMP(t1), e)
    munchExp(src); emit("ADD");
}

void munchMove(Exp dst, Exp src) {
    // MOVE(d, e)
    if (dst instanceof MEM) munchMove((MEM)dst, src);
    else if (dst instanceof TEMP) munchMove((TEMP)dst, src);
}

void munchStm(Stm s) {
    if (s instanceof MOVE) munchMove(((MOVE)s).dst, ((MOVE)s).src);
    : // CALL, JUMP, CJUMP unimplemented here
}
```


9.1 명령어 선택 알고리즘

MEM(...) → 메모리에서 읽기: LOAD
BINOP(PLUS, ...) → 덧셈 계산: ADD 또는 ADDI
CONST(i) → 상수 생성: ADDI 이용
TEMP(t) → 이미 레지스터에 있음: 명령어 없음

```
void munchExp(Exp)
MEM(BINOP(PLUS, e1, CONST(i))) ⇒ munchExp(e1); emit("LOAD");
MEM(BINOP(PLUS, CONST(i), e1)) ⇒ munchExp(e1); emit("LOAD");
MEM(CONST(i)) ⇒ emit("LOAD");
MEM(e1) ⇒ munchExp(e1); emit("LOAD");
BINOP(PLUS, e1, CONST(i)) ⇒ munchExp(e1); emit("ADDI");
BINOP(PLUS, CONST(i), e1) ⇒ munchExp(e1); emit("ADDI");
CONST(i) ⇒ munchExp(e1); emit("ADDI");
BINOP(PLUS, e1, CONST(i)) ⇒ munchExp(e1); emit("ADD");
TEMP(t) ⇒ { }
```

PROGRAM 9.3. Maximal Munch in Java.

9.1 명령어 선택 알고리즘

9.2 CISC 컴퓨터

9.3 MiniJava 컴파일러의 명령어 선택

타일 매칭 이후의 문제: 레지스터는 어떻게 정할 것인가?

- 프로그램 9.3에서 각 패턴이 매칭된 뒤에 단순히 명령어 이름을 출력하는데, 이 명령어들은 어떤 레지스터를 사용하는지 알 수 없음

타일의 루트와 중간 결과

- BINOP(PLUS, e1, e2)가 ADD 타일로 커버되면 (ADD r3, r1, r2) r3가 이 타일 루트의 결과를 담는 레지스터

9.3 MiniJava 컴파일러의 명령어 선택

레지스터 할당(register allocation)

- 앞 슬라이드에서와 같이 각 중간 결과 노드에 실제 레지스터 번호를 배정하는 작업

9.3 MiniJava 컴파일러의 명령어 선택

명령어 선택과 레지스터 할당의 관계

- 명령어 선택 단계에서 레지스터 할당을 동시에 수행할 수 있음
- 즉, 타일을 선택하면서 바로 다음처럼 명령어를 만들 수 있음

```
LOAD r1, 8(fp)
ADD  r2, r1, r3
STORE r2, 12(fp)
```

레지스터 할당의 많은 부분은 특정 기계 명령어 집합과 독립적이므로 각 타깃 기계마다 레지스터 할당 알고리즘을 다시 구현하는 것은 바람직하지 않음

9.3 MiniJava 컴파일러의 명령어 선택

명령어 선택 전 레지스터 할당

- 레지스터가 필요한 것은 각 타일의 루트
- 모든 노드가 레지스터를 필요로 하는 것은 아님
- 아직 어떤 트리 노드가 실제로 레지스터를 필요로 할지 알 수 없음
- 예) MEM (BINOP (PLUS, TEMP fp, CONST 8))

=> LOAD r1, 8(fp)

- 불필요한 노드에도 레지스터 배정될 가능성이 있지만,
- 코드 생성 구현이 단순함

9.3 MiniJava 컴파일러의 명령어 선택

이 교재의 방식: 명령어 선택 이후에 레지스터 할당

- 명령어 선택 단계에서는 정확한 실제 레지스터 번호를 모른 채 명령어를 생성
- 처음에는 임시 레지스터 **t1, t2, t3**를 사용
- 나중에 레지스터 할당 단계에서 임시 레지스터를 실제 레지스터로 바꿈

```
LOAD t1, 8(fp)
ADD  t2, t1, t3
STORE t2, 12(fp)
```



```
LOAD r4, 8(fp)
ADD  r5, r4, r6
STORE r5, 12(fp)
```


9.3 MiniJava 컴파일러의 명령어 선택

Assem (Abstract Assembly Language Instructions)

- 기계어 독립적인 기계어 표현 방법

특정 기계어 명령어를 하나의 클래스로 표현하지 않고,

Instr 클래스의 **String** 타입의 **assem** 필드에 원하는 명령어 이름 문자열을 지정하여 표현

```
package Assem;
import Temp.TempList;

public abstract class Instr {
    public String assem;
    public abstract TempList use();
    public abstract TempList def();
    public abstract Targets jumps();
    public String format(Temp.TempMap m);
}

public Targets(Temp.LabelList labels);

public OPER(String assem, TempList dst, TempList src,
             Temp.LabelList jump);
public OPER(String assem, TempList dst, TempList src);
public MOVE(String assem, Temp dst, Temp src);
public LABEL(String assem, Temp.Label label);
```

li 명령어와 jal 명령어 => OPER 클래스

move 명령어 => MOVE 클래스

9.3 MiniJava 컴파일러의 명령어 선택

Assem (Abstract Assembly Language Instructions)

- 기계어 독립적인 기계어 표현 방법 Tree IR의 문장으로 변환

```
package Assem;
import Temp.TempList;

public abstract class Instr {
    public String assem;
    public abstract TempList use();
    public abstract TempList def();
    public abstract Targets jumps();
    public String format(Temp.TempMap m);
}

public Targets(Temp.LabelList labels);
```

```
public OPER(String assem, TempList dst, TempList src,
            Temp.LabelList jump);
public OPER(String assem, TempList dst, TempList src);
public MOVE(String assem, TempList dst, TempList src);
public LABEL(String assem, Temp.Label label);
```

```
EXP(
    CALL(
        NAME _print,
        CONST 123)),
```

```
new OPER( "li `d0, 123", new TempList(tVal, null), null )
```

String assem TempList dst

TempList src

명령어 선택(Assem)

```
li t36, 123
```

```
move $a0, t36
```

```
jal _print
```

li 명령어와 jal 명령어 => OPER 클래스

move 명령어 => MOVE 클래스

9.3 MiniJava 컴파일러의 명령어 선택

Assem (Abstract Assembly Language Instructions)

- 기계어 독립적인 기계어 표현 방법

```
import Assem.Instr;
import Assem.OPER;
import Temp.Temp;
import Temp.TempList;
import Temp.DefaultMap;
```

```
public abstract class Instr {
    public String assem;
    public abstract TempList use();
    public abstract TempList def();
    public abstract Targets jumps();
    public String format(Temp.TempMap m);
}
```

```
public class OPER extends Instr {
    public Temp.TempList dst;
    public Temp.TempList src;
    public Targets jump;
```

```
public class AssemExample {
    public static void main(String[] args) {
        DefaultMap m = new DefaultMap();
        Temp tVal = new Temp(); // t36
        Temp tA0 = ... // 맵에 의해 $a0인 변수
```

```
Instr i1 = new OPER( "li `d0, 123",    new TempList(tVal, null),    null    );
emit( i1 );
```

```
Instr i2 = new OPER( "move `d0, `s0", new TempList(tA0,null), new TempList(tVal, null) );
emit( i2 );
```

```
String lbl = "_print";
Instr i3 = new OPER(    "jal " + lbl,    null,    null    );
emit( i3 );
```

```
System.out.println(i1.format(m), i2.format(m), i3.format(m) );
```

```
}
```

```
li t36, 123
move $a0, t36
jal _print
```

Assem.Instr.format() 함수 정의 참고

9.3 MiniJava 컴파일러의 명령어 선택

명령어 선택 단계에서 실제 어셈블리 명령어를 바로 문자열로 출력하지 않고, 각 명령어를 객체로 표현

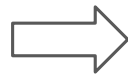
- 명령어 기반 클래스: Instr
- 명령어 종류별 클래스: OPER, LABEL, MOVE

9.3 MiniJava 컴파일러의 명령어 선택

일반 연산 명령어 OPER

- **assem**: 어셈블리 명령어 문자열
- **src** : 피연산자 레지스터 목록
- **dst** : 결과 레지스터 목록

```
ADD t3, t1, t2
```



```
assem = "ADD `d0, `s0, `s1"  
src   = [t1, t2]  
dst   = [t3]
```

9.3 MiniJava 컴파일러의 명령어 선택

분기 명령어 OPER

- **assem**: 어셈블리 명령어 문자열
- **src** : 피연산자 레지스터 목록
- **dst** : 결과 레지스터 목록
- **jump** : 라벨 목록 (true label, false label)

(참고) 다음 명령어로 이어지는 방식(**fall-through**)으로 분기할 경우

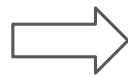
그 다음 명령어의 라벨도 명시적으로 포함

9.3 MiniJava 컴파일러의 명령어 선택

라벨 명령어 LABEL

- **assem**: 어셈블리 명령어 문자열
- **label**: 실제 라벨 문자열

L1:



```
assem = "L1:"  
label = L1
```

9.3 MiniJava 컴파일러의 명령어 선택

데이터 이동 명령어 MOVE

- **assem**: 어셈블리 명령어 문자열

```
MOV t1, t2
```

- **dst** : 타겟 레지스터

- **src** : 소스 레지스터

9.3 MiniJava 컴파일러의 명령어 선택

Instr 클래스의 `use()`와 `def()` 메소드

- `usr()` : `src` 목록, 이 명령어가 읽는 임시 레지스터들을 반환
 - `def()` : `dst` 목록, 이 명령어가 새로 값을 쓰는 임시 레지스터들을 반환
- => 10장 Liveness analysis와 11장 레지스터 할당에 사용

9.3 MiniJava 컴파일러의 명령어 선택

Instr 클래스의 `format(m)` 메소드

- 임시 변수 이름을 실제 레지스터 이름으로 바꾸어 최종 어셈블리 문자열을 만듦
- `TempMap`에서 임시 변수를 레지스터로 매핑

```
package Temp;  
public interface TempMap {  
    public String tempMap(Temp.Temp t);  
}
```

TempMap m

t1 → r1
t2 → r2
t3 → r3

`i.format(m)`

assem = "ADD `d0, `s0, `s1"
src = [t1, t2]
dst = [t3]



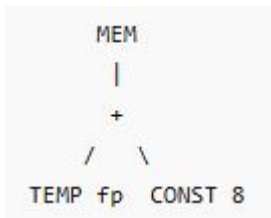
ADD r3, r1, r2

9.3 MiniJava 컴파일러의 명령어 선택

Assem.Instr 클래스는 타깃 기계의 어셈블리 언어와 독립적으로 설계

- 예) 메모리 읽기

MEM(fp + 8)



TempMap

```
frame.FP() → r27  
new Temp() → r1
```

```
new OPER("LOAD `d0 <- M[`s0+8]",  
    new TempList(new Temp(), null),  
    new TempList(frame.FP(), null));
```

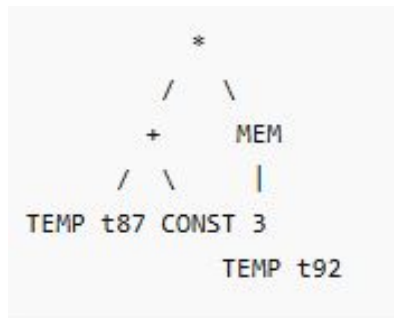


```
LOAD r1 <- M[r27+8]
```

9.3 MiniJava 컴파일러의 명령어 선택

Assem.Instr 클래스는 타깃 기계의 어셈블리 언어와 독립적으로 설계

- 예) 덧셈, 메모리 읽기, 곱셈 $(t87 + 3) * \text{MEM}(t92)$



TempMap

```
t87  → r12
t92  → r13
t908 → r1
t909 → r2
t910 → r1
```

assem	dst	src
ADDI `d0 <- `s0+3	t908	t87
LOAD `d0 <- M[`s0+0]	t909	t92
MUL `d0 <- `s0*`s1	t910	t908, t909



```
ADDI r1 <- r12+3
LOAD r2 <- M[r13+0]
MUL  r1 <- r1*r2
```

(레지스터 할당 이전 단계에서는 TempMap의 임시 변수가 아직 레지스터로 매핑되는 정보가 없을 수 있음.

9.3 MiniJava 컴파일러의 명령어 선택

Instr의 `assem` 문자열의 표기법

- 소스 레지스터: ``s0, `s1, ..., `s(k-1)`

- 목적 레지스터: ``d0, `d1, ...,`

- `MUL `d0 <- `s0*`s1`

``s0` 첫 번째 소스 템프, ``s1` 두 번째 소스 템프

``d0` 첫 번째 목적 템프

``s0`에서 ```은 `s0`를 찾을 때
사용할 구분자

9.3 MiniJava 컴파일러의 명령어 선택

Instr의 `assem` 문자열의 표기법

- 점프 명령어의 라벨: ``j0`, ``j1`, ...
- 라벨 목록에 들어있는 라벨들
- 조건 분기 명령어는 보통 두 개의 라벨을 갖지만, 어셈블리 문자열에는 실제 분기 대상 하나만 나타나고 **'fall-through'** 라벨은 나타나지 않을 수 있음

9.3 MiniJava 컴파일러의 명령어 선택

어셈블리 명령어 만들기

기본 아이디어

- 패턴 매칭 절의 오른쪽 부분을 작성
- IR Tree 식을 커버하는 **munch** 함수들이 그 결과 **Assem** 명령어를 생성

```
Temp.Temp munchExp(Tree.Exp e);  
void      munchStm(Tree.Stm s);
```

9.3 MiniJava 컴파일러의 명령어 선택

munchExp 함수

- 명령어를 생성하고 그 결과가 들어 있는 임시 레지스터를 반환
- Temp.Temp munchExp(Tree.Exp e);
- 예) t2 = munchExp (BINOP(PLUS, TEMP t1, CONST 3))

ADDI t2 <- t1 + 3 어셈블리 명령어 생성

t2를 반환

9.3 MiniJava 컴파일러의 명령어 선택

munchStm 함수

- void munchStm(Treem.Stm s);
- 문장은 보통 값을 반환하지 않음
- 예) munchStm (MOVE(MEM(e1), e2))

STORE 어셈블리 명령어 생성

9.3 MiniJava 컴파일러의 명령어 선택

상향 방식(Bottom-up)

- munchStm과 munchExp는 명령어를 상향 방식으로 생성
- 먼저 하위 식들을 처리하고 올라오면서 명령어를 출력
- 예) BINOP(PLUS, e1, e2)
 - e1을 계산하는 명령어 생성
 - > e2를 계산하는 명령어 생성
 - > 두 결과를 더하는 **ADD** 명령어를 생성

9.3 MiniJava 컴파일러의 명령어 선택

emit 함수의 역할

- 생성된 명령어를 리스트에 차례대로 모아두고 나중에 반환
- 생성된 명령어를 즉시 출력하지 않음

```
emit(instr);
```

```
munchStm(EXP(CALL(e, args)))  
{  
    Temp r = munchExp(e);  
    TempList l = munchArgs(0, args);  
    emit(new OPER("CALL `s0\n", calldefs, L(r, l)));  
}
```

- `munchExp(e)` 안에서 `emit` 호출로 `e`에 대한 명령어를 쌓고
- `munchArgs(0, args)` 안에서 `emit` 호출로 인자에 대한 명령어를 쌓은 다음에,
- `CALL` 명령어를 `emit`으로 추가

9.3 MiniJava 컴파일러의 명령어 선택

```
TempList L(Temp h, TempList t) {return new TempList(h,t);}
```

```
munchStm(SEQ(a,b))
    {munchStm(a); munchStm(b);}
munchStm(MOVE(MEM(BINOP(PLUS,e1,CONST(i))),e2))
    emit(new OPER("STORE M['s0+" + i + "] <- 's1\n",
        null, L(munchExp(e1), L(munchExp(e2), null))));
munchStm(MOVE(MEM(BINOP(PLUS,CONST(i),e1)),e2))
    emit(new OPER("STORE M['s0+" + i + "] <- 's1\n",
        null, L(munchExp(e1), L(munchExp(e2), null))));
munchStm(MOVE(MEM(e1),MEM(e2)))
    emit(new OPER("MOVE M['s0] <- M['s1\n",
        null, L(munchExp(e1), L(munchExp(e2), null))));
munchStm(MOVE(MEM(CONST(i)),e2))
    emit(new OPER("STORE M[r0+" + i + "] <- 's0\n",
        null, L(munchExp(e2), null)));
munchStm(MOVE(MEM(e1),e2))
    emit(new OPER("STORE M['s0] <- 's1\n",
        null, L(munchExp(e1), L(munchExp(e2), null))));
munchStm(MOVE(MEM(i),e2))
    emit(new OPER("ADD 'd0 <- 's0 + r0\n",
        L(i,null), L(munchExp(e2), null)));
munchStm(LABEL(lab))
    emit(new Assem.LABEL(lab.toString() + ":\n", lab));
```

- a에 대한 기계어를 생성하고
- b에 대한 기계어를 생성함
- seq 자체는 별도의 기계어로 번역되지는
않고, 내부의 두 문장을 순서대로 번역하도록
가이드

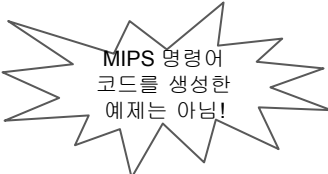
- e1: 주소 기준값을 담은 레지스터를 생성
- e2: 저장할 값을 담은 레지스터를 생성
- null: 이 명령어가 새로 정의하는 레지스터는
없음

- e1: 목적지 주소를 담은 레지스터를 생성
- e2: 원본 주소를 담은 레지스터를 생성
- MIPS는 메모리에서 메모리로 직접 이동
불가. 메모리->레지스터->메모리로 번역

MIPS 명령어
코드를 생성한
예제는 아님!

9.3 MiniJava 컴파일러의 명령어 선택

```
munchExp(MEM(BINOP(PLUS,e1,CONST(i))))
    Temp r = new Temp();
    emit(new OPER("LOAD `d0 <- M[`s0+" + i + "]\n",
                  L(r,null), L(munchExp(e1),null)));
    return r;
munchExp(MEM(BINOP(PLUS,CONST(i),e1)))
    Temp r = new Temp();
    emit(new OPER("LOAD `d0 <- M[`s0+" + i + "]\n",
                  L(r,null), L(munchExp(e1),null)));
    return r;
munchExp(MEM(CONST(i)))
    Temp r = new Temp();
    emit(new OPER("LOAD `d0 <- M[r0+" + i + "]\n",
                  L(r,null), null));
    return r;
munchExp(MEM(e1))
    Temp r = new Temp();
    emit(new OPER("LOAD `d0 <- M[`s0+0]\n",
                  L(r,null), L(munchExp(e1),null)));
    return r;
```

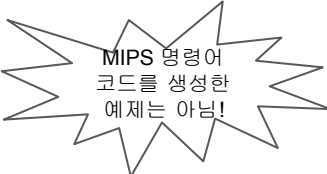


MIPS 명령어
코드를 생성한
예제는 아님!

9.3 MiniJava 컴파일러의 명령어 선택

```
munchExp(BINOP(PLUS,e1,CONST(i)))
    Temp r = new Temp();
    emit(new OPER("ADDI 'd0 <- 's0+" + i + "\n",
                  L(r,null), L(munchExp(e1),null)));
    return r;
munchExp(BINOP(PLUS,CONST(i),e1))
    Temp r = new Temp();
    emit(new OPER("ADDI 'd0 <- 's0+" + i + "\n",
                  L(r,null), L(munchExp(e1),null)));
    return r;
munchExp(CONST(i))
    Temp r = new Temp();
    emit(new OPER("ADDI 'd0 <- r0+" + i + "\n",
                  null, L(munchExp(e1),null)));
    return r;
munchExp(BINOP(PLUS,e1,e2))
    Temp r = new Temp();
    emit(new OPER("ADD 'd0 <- 's0+'s1\n",
                  L(r,null), L(munchExp(e1),L(munchExp(e2),null))));
    return r;
munchExp(TEMP(t))
    return t;
```

PROGRAM 9.6. Assem-instructions for munchExp.



MIPS 명령어
코드를 생성한
예제는 아님!

9.3 MiniJava 컴파일러의 명령어 선택

```
package Jouette;
public class Codegen {
    Frame frame;
    public Codegen(Frame f) {frame=f;}

    private Assem.InstrList  ilist=null, last=null;

    private void emit(Assem.Instr inst) {
        if (last!=null)
            last = last.tail = new Assem.InstrList(inst,null);
        else last = ilist = new Assem.InstrList(inst,null);
    }

    void      munchStm(Tree.Stm s) { ... }
    Temp.Temp munchExp(Tree.Exp s) { ... }

    Assem.InstrList codegen(Tree.Stm s) {
        Assem.InstrList l;
        munchStm(s);
        l=ilist;
        ilist=last=null;
        return l;
    }
}
```

```
package Frame;
public class Frame {
    ...
    public Assem.InstrList codegen(Tree.Stm stm); {
        return (new Codegen(this)).codegen(stm);
    }
}
```

PROGRAM 9.7. The Codegen class.

9.3 MiniJava 컴파일러의 명령어 선택

프로시저 호출 EXP(CALL(f, args)) 또는 MOVE(TEMP t, CALL(f, args))

타일 매칭

```
munchStm(EXP(CALL(e, args)))  
{  
    Temp r = munchExp(e);  
    Templist l = munchArgs(0, args);  
    emit(new OPER("CALL `s0\n", calldefs, L(r, l)));  
}
```

1. 호출할 함수 주소 **e**를 계산
2. 인자 **args**를 올바른 위치에 배치
3. CALL 명령어를 생성

9.3 MiniJava 컴파일러의 명령어 선택

타일 매칭

```
munchStm(EXP(CALL(e, args)))  
{  
    Temp r = munchExp(e);  
    TempList l = munchArgs(0, args);  
    emit(new OPER("CALL `s0\n", calldefs, L(r, l)));  
}
```

- munchArgs는 모든 인자를 올바른 위치로 옮기는 코드를 생성

: i=0 -> 첫 번째 인자

1. 레지스터에 전달될 인자
2. 메모리, 즉 스택의 인자 전달 영역

: i=1 -> 두 번째 인자

: ...

9.3 MiniJava 컴파일러의 명령어 선택

타일 매칭

```
munchStm(EXP(CALL(e, args)))  
{  
    Temp r = munchExp(e);  
    TempList l = munchArgs(0, args);  
    emit(new OPER("CALL `s0\n", calldefs, L(r, l)));  
}
```

- munchArgs는 모든 인자를 올바른 위치로 옮기는 코드를 생성
 - : CALL 명령어에 전달되어야 하는 TempList를 반환
 - : 이 TempList를 CALL 명령어의 소스 리스트에 포함(src())

9.3 MiniJava 컴파일러의 명령어 선택

타일 매칭

```
munchStm(EXP(CALL(e, args)))
{
    Temp r = munchExp(e);
    TempList l = munchArgs(0, args);
    emit(new OPER("CALL `s0\n", calldefs, L(r, l)));
}
```

- munchArgs는 모든 인자를 올바른 위치로 옮기는 코드를 생성

: 함수 호출로 특정 레지스터 값을 변경할 수 있음

: caller-save 레지스터, return-address 레지스터, return-value 레지스터

: 이 레지스터들을 calldefs라고 함, CALL 명령어의 dest 목록에 넣음 (def())

9.3 MiniJava 컴파일러의 명령어 선택

munchArgs 의사 코드

- 0,1,2 인자를 r4, r5, r6 레지스터으로 전달, 그 이후 인자를 스택으로 전달

```
TempList munchArgs(int i, ExpList args) {
    if (args == null) return null;

    Temp argTemp = munchExp(args.head);

    if (i == 0)
        emit(new MOVE(argReg(0), argTemp)); // r4 <- argTemp
    else if (i == 1)
        emit(new MOVE(argReg(1), argTemp)); // r5 <- argTemp
    else if (i == 2)
        emit(new MOVE(argReg(2), argTemp)); // r6 <- argTemp
    else
        emit(new OPER("STORE `s0 -> stack_arg_" + i,
            null,
            L(argTemp, null)));

    return L(argReg(i), munchArgs(i + 1, args.tail));
}
```

9.3 MiniJava 컴파일러의 명령어 선택

프레임 포인터가 없는 경우

[실습]

MIPS를 최종 타겟으로, IR Tree를 Assem 명령어로 번역하는 코드를 구현

- IR Tree

- > Canon 모듈로 정규화

- > MIPS.Codegen에서 maximal munch 적용

- > Assem.Instr 리스트 생성

- > TempMap을 이용해 format -> 레지스터 할당 전 어셈블리 출력

[실습]

- MIPS를 최종 타겟으로, IR Tree를 Assem 명령어로 번역하는 코드를 구현
- Maximal munch 알고리즘을 사용
- MIPS.Codegen 클래스 작성 (Program 9.7 참고)
 - : Tree IR -> Assem instructions
- Codegen 모듈을 적용하기 전 Canon 모듈로 IR 트리를 단순화 시켜야 함
 - : 원래 IR Tree -> Canon 모듈로 정규화 -> Codegen 모듈로 Assem 명령어 생성
- format 함수를 사용하여 실제 어셈블리 언어 문자열로 바꿈
 - : 임시 레지스터를 자신의 문자열 이름으로 (Temp.DefaultMap 교재 참고)

[실습]

- built-in 임시 레지스터

: **Frame.FP** (프레임 포인터 레지스터)

: **Frame** 모듈에서 **Frame.FP** -> **fp**로 변환하는 매핑 제공

```
package Frame;  
  
public class Frame implements Temp.TempMap {  
    ...  
    abstract public String tempMap(Temp temp);  
}
```

1. 특별한 이름 -> 해당 기계 레지스터 이름
2. 일반 이름 -> null 반환

- 레지스터 할당 전 출력용 **TempMap** (널이 아니면 그대로, 널이면, 임시 이름)

[실습]

CodeGen.

CodeGen

클래스 구성

```
1 package Codegen;
2
3 import Assem.*;
4 import Temp.*;
5 import Tree.*;
6 import Mips.Frame;
7 import java.util.ArrayList;
8
9 /**
10  * MIPS-oriented Maximal Munch code generator.
11  * - Consumes trace-scheduled IR (ESEQ-free, CALLs in statements or MOVE(TEMP, CALL)).
12  * - Emits Assem.InstrList using simple MIPS (and pseudo) opcodes.
13  * - No calling convention or register allocation: temps print as tN via DefaultMap.
14  */
15 public class Codegen {
16     private InstrList head, last;
17
18     private void emit(Instr i) { ...
19
20
21
22
23     public InstrList codegen(Tree.StmList stms) { ...
24
25
26
27
28     private void munchStm(Tree.Stm s) { ...
29
30
31
32
33
34
35     private static class Addr { ...
36
37
38
39
40
41
42     private Addr munchAddr(Tree.Exp e) { ...
43
44
45
46
47
48
49     private String branchOp(int rel) { ...
50
51
52
53
54
55
56     private Temp munchExp(Tree.Exp e) { ...
57
58
59
60
61
62
63     private Temp munchCall(Tree.CALL call, Temp resOpt) { ...
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
```

[실습]

CodeGen.emit(Instr i)

```
15 public class Codegen {  
16     private InstrList head, last;  
17  
18     private void emit(Instr i) {  
19         if (last == null) head = last = new InstrList(i, null);  
20         else last = last.tail = new InstrList(i, null);  
21     }
```

[실습]

CodeGen.codeGen(Tree.StmList stms)

```
23 | public InstrList codegen(Tree.StmList stms) {  
24 |     head = last = null;  
25 |     for (Tree.StmList l = stms; l != null; l = l.tail) {  
26 |         munchStm(l.head);  
27 |     }  
28 |     return head;  
29 | }
```

[실습]

CodeGen.munchStm(Tree.Stm s)

```
31 private void munchStm(Tree.Stm s) {  
32 >     if (s instanceof Tree.LABEL) { ...  
35 >     } else if (s instanceof Tree.JUMP) { ...  
44 >     } else if (s instanceof Tree.CJUMP) { ...  
54 >     } else if (s instanceof Tree.MOVE) { ...  
82 >     } else if (s instanceof Tree.EXP) { ...  
89     } else {  
90         // Should not happen after canonicalization, but handle defensively  
91         throw new Error("Unsupported Stm in Codegen: " + s.getClass());  
92     }  
93 }
```

[실습]

CodeGen.munchStm(Tree.Stm s)

```
31  > private void munchStm(Tree.Stm s) {  
32  >     if (s instanceof Tree.LABEL) {  
33  >         Tree.LABEL lab = (Tree.LABEL) s;  
34  >         emit(new Assem.LABEL(lab.label.toString() + ":", lab.label));  
35  >     } else if (s instanceof Tree.JUMP) { ...
```

[실습]

CodeGen.munchStm(Tree.Stm s)

```
31 private void munchStm(Tree.Stm s) {  
32 >   if (s instanceof Tree.LABEL) { ...  
35     } else if (s instanceof Tree.JUMP) {  
36       Tree.JUMP j = (Tree.JUMP) s;  
37       if (j.exp instanceof Tree.NAME) {  
38         Label target = ((Tree.NAME) j.exp).label;  
39         emit(new OPER(a: "j `j0", d: null, s: null, new LabelList(target, null)));  
40       } else {  
41         Temp t = munchExp(j.exp);  
42         emit(new OPER(a: "jr `s0", d: null, new TempList(t, null)));  
43       }  
44 >   } else if (s instanceof Tree.CJUMP) { ...
```

[실습]

CodeGen.munchStm(Tree.Stm s)

```
31 private void munchStm(Tree.Stm s) {  
32 >   if (s instanceof Tree.LABEL) { ...  
35 >   } else if (s instanceof Tree.JUMP) { ...  
44   } else if (s instanceof Tree.CJUMP) {  
45       Tree.CJUMP c = (Tree.CJUMP) s;  
46       Temp a = munchExp(c.left);  
47       Temp b = munchExp(c.right);  
48       String br = branchOp(c.relop);  
49       emit(new OPER(br + " `s0, `s1, `j0",  
50           d: null,  
51           new TempList(a, new TempList(b, null)),  
52           new LabelList(c.iftrue, null)));  
53       // fall-through to c.iffalse is handled by trace scheduling / block layout  
54 >   } else if (s instanceof Tree.MOVE) { ...
```

[실습]

CodeGen.munchStm(Tree.Stm s)

```
31 private void munchStm(Tree.Stm s) {
32 >   if (s instanceof Tree.LABEL) { ...
35 >   } else if (s instanceof Tree.JUMP) { ...
44 >   } else if (s instanceof Tree.CJUMP) { ...
54   } else if (s instanceof Tree.MOVE) {
55       Tree.MOVE m = (Tree.MOVE) s;
56       if (m.dst instanceof Tree.TEMP && m.src instanceof Tree.TEMP) {
57           Temp d = ((Tree.TEMP) m.dst).temp;
58           Temp x = ((Tree.TEMP) m.src).temp;
59           emit(new Assem.MOVE(a: "move `d0, `s0", d, x));
60 >       } else if (m.dst instanceof Tree.TEMP && m.src instanceof Tree.CONST) { ...
64 >       } else if (m.dst instanceof Tree.TEMP && m.src instanceof Tree.MEM) { ...
69 >       } else if (m.dst instanceof Tree.MEM) { ...
74 >       } else if (m.dst instanceof Tree.TEMP) { ...
78       } else {
79           // Side-effect-only exp
80           munchExp(m.src);
81       }
82 >   } else if (s instanceof Tree.EXP) { ...
```


[실습]

CodeGen.munchStm(Tree.Stm s)

```
31 private void munchStm(Tree.Stm s) {
32 >   if (s instanceof Tree.LABEL) { ...
35 >   } else if (s instanceof Tree.JUMP) { ...
44 >   } else if (s instanceof Tree.CJUMP) { ...
54   } else if (s instanceof Tree.MOVE) {
55       Tree.MOVE m = (Tree.MOVE) s;
56 >       if (m.dst instanceof Tree.TEMP && m.src instanceof Tree.TEMP) { ...
60 >       } else if (m.dst instanceof Tree.TEMP && m.src instanceof Tree.CONST) { ...
64 >       } else if (m.dst instanceof Tree.TEMP && m.src instanceof Tree.MEM) { ...
69 >       } else if (m.dst instanceof Tree.MEM) { ...
74       } else if (m.dst instanceof Tree.TEMP) {
75           Temp d = ((Tree.TEMP) m.dst).temp;
76           Temp v = munchExp(m.src);
77           emit(new Assem.MOVE(a: "move `d0, `s0", d, v));
78       } else {
79           // Side-effect-only exp
80           munchExp(m.src);
81       }
82 >   } else if (s instanceof Tree.EXP) { ...
89 >   } else { ...
93 }
```

[실습]

CodeGen.munchStm(Tree.Stm s)

```
31 private void munchStm(Tree.Stm s) {  
32 >     if (s instanceof Tree.LABEL) { ...  
35 >     } else if (s instanceof Tree.JUMP) { ...  
44 >     } else if (s instanceof Tree.CJUMP) { ...  
54 >     } else if (s instanceof Tree.MOVE) { ...  
82     } else if (s instanceof Tree.EXP) {  
83         Tree.EXP e = (Tree.EXP) s;  
84         if (e.exp instanceof Tree.CALL) {  
85             munchCall((Tree.CALL) e.exp, null);  
86         } else {  
87             munchExp(e.exp);  
88         }  
89 >     } else { ...  
93 }
```

[실습]

CodeGen.munchExp(Tree.Exp e)

```
133 private Temp munchExp(Tree.Exp e) {  
134 >   if (e instanceof Tree.TEMP) { ...  
136 >   } else if (e instanceof Tree.CONST) { ...  
141 >   } else if (e instanceof Tree.NAME) { ...  
146 >   } else if (e instanceof Tree.MEM) { ...  
151 >   } else if (e instanceof Tree.BINOP) { ...  
185 >   } else if (e instanceof Tree.CALL) { ...  
187 >   } else if (e instanceof Tree.ESEQ) { ...  
192   } else {  
193     throw new Error("Unsupported Exp in Codegen: " + e.getClass());  
194   }  
195 }
```

[실습]

CodeGen.munchExp(Tree.Exp e)

```
133     private Temp munchExp(Tree.Exp e) {  
134         if (e instanceof Tree.TEMP) {  
135             return ((Tree.TEMP) e).temp;  
136 >     } else if (e instanceof Tree.CONST) { ...
```

[실습]

CodeGen.munchExp(Tree.Exp e)

```
133     private Temp munchExp(Tree.Exp e) {  
134 >         if (e instanceof Tree.TEMP) { ...  
136             } else if (e instanceof Tree.CONST) {  
137                 Temp t = new Temp();  
138                 int k = ((Tree.CONST) e).value;  
139                 emit(new OPER("li `d0, " + k, new TempList(t, null), s: null));  
140                 return t;  
141 >             } else if (e instanceof Tree.NAME) { ...
```

[실습]

CodeGen.munchExp(Tree.Exp e)

```
133     private Temp munchExp(Tree.Exp e) {  
134 >         if (e instanceof Tree.TEMP) { ...  
136 >     } else if (e instanceof Tree.CONST) { ...  
141         } else if (e instanceof Tree.NAME) {  
142             Temp t = new Temp();  
143             String lbl = ((Tree.NAME) e).label.toString();  
144             emit(new OPER("la `d0, " + lbl, new TempList(t, null), s: null));  
145             return t;  
146 >     } else if (e instanceof Tree.MEM) { ...
```

[실습]

CodeGen.munchExp(Tree.Exp e)

```
133 private Temp munchExp(Tree.Exp e) {  
134 >     if (e instanceof Tree.TEMP) { ...  
136 >     } else if (e instanceof Tree.CONST) { ...  
141 >     } else if (e instanceof Tree.NAME) { ...  
146     } else if (e instanceof Tree.MEM) {  
147         Temp t = new Temp();  
148         Addr a = munchAddr(((Tree.MEM) e).exp);  
149         emit(new OPER("lw `d0, " + a.off + "("`s0)", new TempList(t, null), new TempList(a.base, null)));  
150         return t;  
151 >     } else if (e instanceof Tree.BINOP) { ...
```

[실습]

CodeGen.munc

```
133 private Temp munchExp(Tree.Exp e) {
134 >     if (e instanceof Tree.TEMP) { ...
136 >     } else if (e instanceof Tree.CONST) { ...
141 >     } else if (e instanceof Tree.NAME) { ...
146 >     } else if (e instanceof Tree.MEM) { ...
151     } else if (e instanceof Tree.BINOP) {
152         Tree.BINOP b = (Tree.BINOP) e;
153         Temp t = new Temp();
154 >         if (b.binop == Tree.BINOP.PLUS && b.right instanceof Tree.CONST) { ...
159 >         } else if (b.binop == Tree.BINOP.PLUS && b.left instanceof Tree.CONST) { ...
164         } else {
165             Temp a = munchExp(b.left);
166             Temp c = munchExp(b.right);
167             String op;
168             switch (b.binop) {
169                 case Tree.BINOP.PLUS: op = "add"; break;
170                 case Tree.BINOP.MINUS: op = "sub"; break;
171                 case Tree.BINOP.MUL: op = "mul"; break; // pseudo
172                 case Tree.BINOP.DIV: op = "div"; break; // pseudo (divu if needed)
173                 case Tree.BINOP.AND: op = "and"; break;
174                 case Tree.BINOP.OR: op = "or"; break;
175                 case Tree.BINOP.XOR: op = "xor"; break;
176                 case Tree.BINOP.LSHIFT: op = "sllv"; break;
177                 case Tree.BINOP.RSHIFT: op = "srlv"; break;
178                 case Tree.BINOP.ARSHIFT: op = "srav"; break;
179                 default: throw new Error("unknown binop");
180             }
181             emit(new OPER(op + " `d0, `s0, `s1",
182                 new TempList(t, null), new TempList(a, new TempList(c, null))));
183             return t;
184         }
185 >     } else if (e instanceof Tree.CALL) { ...
```


[실습]

CodeGen.munchExp(Tree.Exp e)

```
133 < private Temp munchExp(Tree.Exp e) {  
134 >     if (e instanceof Tree.TEMP) { ...  
136 >     } else if (e instanceof Tree.CONST) { ...  
141 >     } else if (e instanceof Tree.NAME) { ...  
146 >     } else if (e instanceof Tree.MEM) { ...  
151 >     } else if (e instanceof Tree.BINOP) { ...  
185 <     } else if (e instanceof Tree.CALL) {  
186     return munchCall((Tree.CALL) e, new Temp());  
187 >     } else if (e instanceof Tree.ESEQ) { ...
```

[실습]

CodeGen.munchExp(Tree.Exp e)

```
133 private Temp munchExp(Tree.Exp e) {  
134 >   if (e instanceof Tree.TEMP) { ...  
136 >   } else if (e instanceof Tree.CONST) { ...  
141 >   } else if (e instanceof Tree.NAME) { ...  
146 >   } else if (e instanceof Tree.MEM) { ...  
151 >   } else if (e instanceof Tree.BINOP) { ...  
185 >   } else if (e instanceof Tree.CALL) { ...  
187   } else if (e instanceof Tree.ESEQ) {  
188     // Should be eliminated by Canon, but handle defensively  
189     Tree.ESEQ x = (Tree.ESEQ) e;  
190     munchStm(x.stm);  
191     return munchExp(x.exp);  
192   } else {
```

[실습]

CodeGen.

munchCall(

Tree.CALL call,

Temp resOpt)

```
197 private Temp munchCall(Tree.CALL call, Temp resOpt) {
198     ArrayList<Temp> argTemps = new ArrayList<>();
199     > for (Tree.ExpList a = call.args; a != null; a = a.tail) { ...
202     int regArgs = Math.min(argTemps.size(), Frame.numArgRegs());
203     int stackArgs = argTemps.size() - regArgs;
204     int stackAdjustment = stackArgs * Frame.wordSize();
205     > if (stackAdjustment > 0) { ...
209     > for (int i = 0; i < regArgs; i++) { ...
214     > for (int i = 0; i < stackArgs; i++) { ...
221
222     TempList callDefs = Frame.callerSaves();
223     > for (int i = Frame.numArgRegs() - 1; i >= 0; i--) { ...
226     callDefs = new TempList(Frame.RV2, callDefs);
227     callDefs = new TempList(Frame.RV, callDefs);
228     callDefs = new TempList(Frame.RA, callDefs);
229
230     TempList callUses = null;
231     > for (int i = regArgs - 1; i >= 0; i--) { ...
234
235     > if (call.func instanceof Tree.NAME) { ...
238     > } else { ...
243
244     > if (stackAdjustment > 0) { ...
248
249     > if (resOpt != null) { ...
252     > } else { ...
255 }
```

[실습]

CodeGen.

munchCall(
Tree.CALL call,
Temp resOpt)

```
197 private Temp munchCall(Tree.CALL call, Temp resOpt) {  
198     ArrayList<Temp> argTemps = new ArrayList<>();  
199 >     for (Tree.ExpList a = call.args; a != null; a = a.tail) { ...  
202         int regArgs = Math.min(argTemps.size(), Frame.numArgRegs());  
203         int stackArgs = argTemps.size() - regArgs;  
204         int stackAdjustment = stackArgs * Frame.wordSize();  
205 >         if (stackAdjustment > 0) { ...  
209 >         for (int i = 0; i < regArgs; i++) { ...  
214 >         for (int i = 0; i < stackArgs; i++) { ...  
221  
222         TempList callDefs = Frame.callerSaves();  
223 >         for (int i = Frame.numArgRegs() - 1; i >= 0; i--) { ...  
226             callDefs = new TempList(Frame.RV2, callDefs);  
227             callDefs = new TempList(Frame.RV, callDefs);  
228             callDefs = new TempList(Frame.RA, callDefs);  
229  
230             TempList callUses = null;  
231 >             for (int i = regArgs - 1; i >= 0; i--) { ...  
234  
235 >             if (call.func instanceof Tree.NAME) { ...  
238 >             } else { ...  
243  
244 >             if (stackAdjustment > 0) { ...  
248  
249 >             if (resOpt != null) { ...  
252 >             } else { ...  
255     }
```

[실습]

CodeGen.

munchCall(

Tree.CALL call,

Temp resOpt)

```
197 private Temp munchCall(Tree.CALL call, Temp resOpt) {
198     ArrayList<Temp> argTemps = new ArrayList<>();
199     for (Tree.ExpList a = call.args; a != null; a = a.tail) {
200         argTemps.add(munchExp(a.head));
201     }
202     int regArgs = Math.min(argTemps.size(), Frame.numArgRegs());
203     int stackArgs = argTemps.size() - regArgs;
204     int stackAdjustment = stackArgs * Frame.wordSize();
205     if (stackAdjustment > 0) {
206         TempList spList = new TempList(Frame.SP, null);
207         emit(new OPER("addi `d0, `s0, -" + stackAdjustment, spList, spList));
208     }
```

[실습]

CodeGen.

munchCall(

Tree.CALL call,

Temp resOpt)

```
197 private Temp munchCall(Tree.CALL call, Temp resOpt) {
198     ArrayList<Temp> argTemps = new ArrayList<>();
199     > for (Tree.ExpList a = call.args; a != null; a = a.tail) { ...
202         int regArgs = Math.min(argTemps.size(), Frame.numArgRegs());
203         int stackArgs = argTemps.size() - regArgs;
204         int stackAdjustment = stackArgs * Frame.wordSize();
205     > if (stackAdjustment > 0) { ...
209         for (int i = 0; i < regArgs; i++) {
210             Temp src = argTemps.get(i);
211             Temp dst = Frame.argReg(i);
212             emit(new Assem.MOVE(a: "move `d0, `s0", dst, src));
213         }
214         for (int i = 0; i < stackArgs; i++) {
215             Temp src = argTemps.get(regArgs + i);
216             int offset = i * Frame.wordSize();
217             emit(new OPER("sw `s0, " + offset + "(`s1)",
218                 d: null,
219                 new TempList(src, new TempList(Frame.SP, null))));
220         }
221     }
```


[실습]

CodeGen.

munchCall(

Tree.CALL call,

Temp resOpt)

```
197 private Temp munchCall(Tree.CALL call, Temp resOpt) {
198     ArrayList<Temp> argTemps = new ArrayList<>();
199 >     for (Tree.ExplList a = call.args; a != null; a = a.tail) { ...
202         int regArgs = Math.min(argTemps.size(), Frame.numArgRegs());
203         int stackArgs = argTemps.size() - regArgs;
204         int stackAdjustment = stackArgs * Frame.wordSize();
205 >         if (stackAdjustment > 0) { ...
209 >         for (int i = 0; i < regArgs; i++) { ...
214 >         for (int i = 0; i < stackArgs; i++) { ...
221
222         TempList callDefs = Frame.callerSaves();
223         for (int i = Frame.numArgRegs() - 1; i >= 0; i--) {
224             callDefs = new TempList(Frame.argReg(i), callDefs);
225         }
226         callDefs = new TempList(Frame.RV2, callDefs);
227         callDefs = new TempList(Frame.RV, callDefs);
228         callDefs = new TempList(Frame.RA, callDefs);
229
230         TempList callUses = null;
231         for (int i = regArgs - 1; i >= 0; i--) {
232             callUses = new TempList(Frame.argReg(i), callUses);
233         }
```

[실습]

CodeGen.

munchCall(

Tree.CALL call,

Temp resOpt)

```
197 private Temp munchCall(Tree.CALL call, Temp resOpt) {
198     ArrayList<Temp> argTemps = new ArrayList<>();
199 > for (Tree.ExpList a = call.args; a != null; a = a.tail) { ...
202     int regArgs = Math.min(argTemps.size(), Frame.numArgRegs());
203     int stackArgs = argTemps.size() - regArgs;
204     int stackAdjustment = stackArgs * Frame.wordSize();
205 > if (stackAdjustment > 0) { ...
209 > for (int i = 0; i < regArgs; i++) { ...
214 > for (int i = 0; i < stackArgs; i++) { ...
221
222     TempList callDefs = Frame.callerSaves();
223 > ⚡ for ((int i = Frame.numArgRegs() - 1; i >= 0; i--)) { ...
226     callDefs = new TempList(Frame.RV2, callDefs);
227     callDefs = new TempList(Frame.RV, callDefs);
228     callDefs = new TempList(Frame.RA, callDefs);
229
230     TempList callUses = null;
231 > for (int i = regArgs - 1; i >= 0; i--) { ...
234
235     if (call.func instanceof Tree.NAME) {
236         String lbl = ((Tree.NAME) call.func).label.toString();
237         emit(new OPER("jal " + lbl, callDefs, callUses));
238     } else {
239         Temp f = munchExp(call.func);
240         callUses = new TempList(f, callUses);
241         emit(new OPER(a: "jalr `s0", callDefs, callUses));
242     }
}
```


[실습]

CodeGen.

munchCall(

Tree.CALL call,

Temp resOpt)

```
197 private Temp munchCall(Tree.CALL call, Temp resOpt) {
198     ArrayList<Temp> argTemps = new ArrayList<>();
199 >     for (Tree.ExpList a = call.args; a != null; a = a.tail) { ...
202         int regArgs = Math.min(argTemps.size(), Frame.numArgRegs());
203         int stackArgs = argTemps.size() - regArgs;
204         int stackAdjustment = stackArgs * Frame.wordSize();
205 >         if (stackAdjustment > 0) { ...
209 >         for (int i = 0; i < regArgs; i++) { ...
214 >         for (int i = 0; i < stackArgs; i++) { ...
221
222         TempList callDefs = Frame.callerSaves();
223 >         for ([int] i = Frame.numArgRegs() - 1; i >= 0; i--) { ...
226             callDefs = new TempList(Frame.RV2, callDefs);
227             callDefs = new TempList(Frame.RV, callDefs);
228             callDefs = new TempList(Frame.RA, callDefs);
229
230         TempList callUses = null;
231 >         for (int i = regArgs - 1; i >= 0; i--) { ...
234
235 >         if (call.func instanceof Tree.NAME) { ...
238 >         } else { ...
243
244         if (stackAdjustment > 0) {
245             TempList spList = new TempList(Frame.SP, null);
246             emit(new OPER("addi `d0, `s0, " + stackAdjustment, spList, spList));
247         }
248
249         if (resOpt != null) {
250             emit(new Assem.MOVE(a: "move `d0, `s0", resOpt, Frame.RV));
251             return resOpt;
252         } else {
253             return null;
254         }
255     }
```

[실습]

CodeGen.Addr 클래스와 CodeGen.munchAddr(Tree.Exp e)

```
95 | private static class Addr {  
96 |     final Temp base;  
97 |     final int off;  
98 |     Addr(Temp b, int o){base=b;off=o;}  
99 | }
```

```
101 | private Addr munchAddr(Tree.Exp e) {  
102 |     if (e instanceof Tree.BINOP) {  
103 |         Tree.BINOP b = (Tree.BINOP) e;  
104 |         if (b.binop == Tree.BINOP.PLUS) {  
105 |             if (b.left instanceof Tree.CONST) {  
106 |                 int k = ((Tree.CONST) b.left).value;  
107 |                 return new Addr(munchExp(b.right), k);  
108 |             } else if (b.right instanceof Tree.CONST) {  
109 |                 int k = ((Tree.CONST) b.right).value;  
110 |                 return new Addr(munchExp(b.left), k);  
111 |             }  
112 |         }  
113 |     }  
114 |     return new Addr(munchExp(e), 0);  
115 | }
```

[실습]

CodeGen.branchOp(int rel)

```
117 private String branchOp(int rel) {  
118     switch (rel) {  
119         case Tree.CJUMP.EQ: return "beq";  
120         case Tree.CJUMP.NE: return "bne";  
121         case Tree.CJUMP.LT: return "blt"; // pseudo  
122         case Tree.CJUMP.GT: return "bgt"; // pseudo  
123         case Tree.CJUMP.LE: return "ble"; // pseudo  
124         case Tree.CJUMP.GE: return "bge"; // pseudo  
125         case Tree.CJUMP.ULT: return "bltu"; // pseudo  
126         case Tree.CJUMP.ULE: return "bleu"; // pseudo  
127         case Tree.CJUMP.UGT: return "bgtu"; // pseudo  
128         case Tree.CJUMP.UGE: return "bgeu"; // pseudo  
129         default: throw new Error("unknown relop");  
130     }  
131 }
```